TRIP PLANNER PWA · FICHE MODULE · MEMBRE 7

Module 7 — Gestion du forum communautaire

Ce que fait le module, la logique métier qu'il porte, et ce que vous devez savoir présenter.

COLLECTION MONGODB	FONCTIONNALITÉ AVANCÉE	RÔLE TRANSVERSE
forum_posts (+ forum_replies)	Recherche plein texte et notifications Web Push	Design system et composants partagés

1. Objectif du module

Offrir un espace public où les voyageurs posent des questions sur des destinations, partagent des conseils et des récits, se répondent, votent et signalent les abus, avec une modération par les administrateurs. C'est la seule partie publique de l'application : elle lui donne de la valeur même en dehors d'un voyage en cours.

> Règle d'équipe : vous êtes le seul à écrire dans votre collection. Les autres modules passent par votre API (ou votre service), et toute modification d'un modèle partagé demande l'accord du groupe.

2. Modèle de données

Champ	Type	Règle / rôle
`forum_posts.title`	texte	10 à 120 caractères
`body`	texte	20 à 5000 caractères
`category`	enum	destinations, budget and tips, transport, accommodation, food, safety, equipment, trip reports
`destinationTags[]`	textes	1 à 5, en minuscules, sans doublon
`author`	ref users | null	null = compte supprimé (anonymisé)
`status, isLocked`	published | hidden, booléen	modération
`upvotes, voters[]`	nombre, refs	un vote par membre
`replyCount, lastActivityAt`	nombre, date	tri « actifs »
`followers[]`	refs users	abonnés aux notifications
`reports[], openReports`	objets, nombre	signalements { reporter, reason, resolved }
`editedAt`	date | null	marqueur « modifié »
`forum_replies.post, parentReply`	refs	parentReply null = réponse ; sinon commentaire
`body`	texte	réponse ≤ 5000, commentaire ≤ 500

3. Logique métier

Lecture et recherche

- Lecture **publique** : les visiteurs consultent sans compte.

- Un post masqué disparaît des listes et n'est visible que par son auteur et les administrateurs.
- **Recherche plein texte** sur le titre (poids 3) et le corps (poids 1) grâce à un **index texte MongoDB** ; filtres par catégorie et par tag de destination.
- Tris : pertinence, récents, les plus votés, récemment actifs. Liste paginée.
- Les tags populaires sont calculés par agrégation.

Publication et discussion

- Écrire demande un compte et une **connexion réseau** (modération et notifications se font sur le serveur) : limite assumée, à expliquer à la soutenance.
- L'auteur **suit automatiquement** son post.
- **Modification** réservée à l'auteur. Après **30 minutes**, une modification affiche le marqueur « edited ». Un fil verrouillé ne peut plus être modifié ni recevoir de réponse.
- **Réponses** : une par minute et par membre (429 sinon).
- **Commentaires** : un seul niveau (un commentaire répond à une réponse, pas à un commentaire), 500 caractères maximum.
- Répondre met à jour `replyCount` et `lastActivityAt`, et abonne le membre au fil, sauf s'il l'a mis en sourdine.
- **Votes** : un par membre et par élément, annulable (bascule atomique côté base).
- **Signalement** : un signalement ouvert par membre et par élément ; il envoie l'élément dans la file de modération.
- **Suppression** par l'auteur ou un administrateur ; supprimer une réponse supprime ses commentaires et corrige le compteur.

Modération (administrateurs)

- File des posts et réponses signalés, avec les motifs et leurs auteurs.
- Actions : masquer / réafficher, verrouiller / déverrouiller le fil, supprimer, ignorer les signalements.
- Un post masqué reste en base avec `status = hidden` (exigence du cahier des charges). Toute action clôt les signalements.

4. Fonctionnalité avancée — Recherche plein texte et notifications Web Push

Recherche plein texte : index `$text` pondéré, tri par score de pertinence, combinable avec les filtres catégorie et tag.

Notifications Web Push (clés VAPID) : à chaque nouvelle réponse, une notification est envoyée à tous les abonnés du fil, sauf à l'auteur de la réponse, à ceux qui ont désactivé les notifications du forum et à ceux qui ont mis ce fil en sourdine.

Un utilisateur peut avoir **plusieurs appareils**. Les abonnements expirés (réponses 404 ou 410 du service push) sont supprimés automatiquement.

« Unfollow » = **sourdine du fil** : on ne reçoit plus rien et on n'est plus abonné automatiquement en répondant. Un réglage global existe aussi dans le profil.

La **permission est demandée en contexte** (après avoir publié, répondu ou suivi un fil), **jamais au premier chargement**.

Le service worker affiche la notification ; un clic ouvre directement la bonne réponse.

Hors ligne : lecture des **50 derniers posts ouverts** (cache du service worker) ; écriture impossible.

5. API REST (préfixe `/api/v1`, réponse `{ data, message, errors }`)

Méthode	Route	Accès	Rôle
GET	`/forum/posts?q&category&tag&sort&page`	public	recherche et liste
GET	`/forum/posts/:postId`	public	post + réponses + commentaires
GET	`/forum/tags, /forum/categories`	public	filtres
POST / PATCH / DELETE	`/forum/posts(/:postId)`	membre / auteur / admin	publier, modifier, supprimer
POST	`/forum/posts/:postId/vote | report`	membre	voter, signaler
POST / DELETE	`/forum/posts/:postId/follow`	membre	suivre / mettre en sourdine
POST	`/forum/posts/:postId/replies`	membre	répondre ou commenter (parentReply)
PATCH / DELETE	`/forum/replies/:replyId`	auteur / admin	modifier, supprimer
POST	`/forum/replies/:replyId/vote | report`	membre	voter, signaler
GET	`/forum/moderation`	admin	file des signalements
POST	`/forum/moderation/posts|replies/:id`	admin	hide, unhide, lock, unlock, delete, dismiss

6. Écrans (front Next.js)

- Liste du forum : recherche, catégories, tri, tags populaires, pagination
- Page d'un fil : votes, suivre, signaler, modifier, réponses et commentaires, demande de permission pour les notifications
- Nouveau post ; onglet « Moderation queue » de l'administration

7. Fichiers dont vous êtes responsable

API — trip-planner-api

- `src/modules/forum/ (forum.model, forum.routes)`
- `src/lib/push.ts`

Front — trip-planner-web

- `src/app/forum/*, src/components/forum/* (PostCard, PostEditor, PushOptIn)`
- `src/app/admin (onglet Moderation)`
- `src/lib/push.ts ; gestion push et notificationclick dans public/sw.js`
- `Design system : src/components/ui.tsx, src/app/globals.css`

8. Liens avec les autres modules

- **Module 1** : comptes, rôle admin, abonnements push et préférences ; anonymisation à la suppression d'un compte.

- **Module 2** : le service worker (cache des posts et affichage des notifications).
- **Tous les modules** : composants partagés du design system (boutons, formulaires, fenêtres, états vides).

9. À démontrer à la soutenance

- ☐ Rechercher « lisbon », filtrer par catégorie et par tag.
- ☐ Publier un post, activer les notifications ; depuis un 2e compte, répondre : la notification arrive.
- ☐ Voter, commenter, tenter deux réponses en moins d'une minute (message de limite).
- ☐ Signaler ; en admin, masquer puis verrouiller : le post disparaît de la liste.
- ☐ Hors ligne : rouvrir un post déjà consulté.

10. Reste à faire

- Tests d'intégration (limite de réponses, modération, recherche) ; requêtes Postman.
- Accessibilité WCAG 2.1 AA (contrastes, clavier, focus) sur l'ensemble du design system.
- Répéter la démo push sur la machine et le navigateur de la soutenance (risque identifié).

11. Questions probables du jury

Pourquoi ne pas autoriser l'écriture hors ligne sur le forum ?
La modération et les notifications dépendent du serveur ; publier plus tard un message hors contexte serait trompeur.

Comment éviter les doublons de votes ?
Le vote est une bascule atomique en base : on ajoute le membre seulement s'il n'a pas déjà voté, sinon on le retire.